

La formation commence par Ruby seul, sans Rails : des exercices en ligne de commande, qui s'enchaînent de 01 à 06. Ce document dit comment installer Ruby, récupérer les supports et faire les exercices.

♦ Installer Ruby

Installez Ruby **4.0**, dans sa dernière version, avec un gestionnaire de versions plutôt qu'avec le Ruby du système. Par exemple avec [mise](#). Sous Windows, travaillez dans **WSL2** (Ubuntu) : toutes les commandes se tapent dans son terminal.

```
curl https://mise.run | sh # installe mise
echo 'eval "$(~/local/bin/mise activate zsh)"' >> ~/.zshrc # sous macOS (zsh)
echo 'eval "$(~/local/bin/mise activate bash)"' >> ~/.bashrc # ou sous Ubuntu et WSL2 (bash)
```

Ouvrez un nouveau terminal, puis :

```
mise use --global ruby@4.0 # installe la dernière version 4.0
ruby -v # affiche ruby 4.0.x
```

Si `ruby -v` affiche une autre version, c'est le Ruby du système qui répond : la ligne d'activation n'est pas prise en compte.

`mise` installe un Ruby précompilé quand il en existe un pour votre machine (Mac Apple Silicon, Linux x86_64). Sinon, il le compile, et il lui faut les outils de compilation :

- Ubuntu : `sudo apt install build-essential autoconf libssl-dev libyaml-dev zlib1g-dev libffi-dev libgmp-dev`
- macOS : `xcode-select --install`, puis `brew install openssl@3 readline libyaml gmp autoconf`

Comme éditeur : VS Code avec l'extension **Ruby LSP**, ou votre éditeur habituel.

Si une commande échoue, gardez le message d'erreur complet : c'est tout ce qu'il faut pour la régler vite.

♦ Récupérer les supports

```
git clone https://github.com/Tuxii/formation-ruby.git
cd formation-ruby
```

```
formation-ruby/
pdf/           les supports : ce mode d'emploi, puis 1-ruby.pdf, le cours
ruby/exercices/ une consigne .md et un squelette .rb à compléter par exercice
ruby/corrections/ les corrections, publiées après chaque exercice
```

Les supports et les corrections arrivent au fil de la formation : `git pull` dans `formation-ruby/` récupère les nouveaux.

♦ Faire un exercice

1. Dans `ruby/exercices/`, ouvrez la consigne, par exemple `01_bases.md`. Son en-tête dit quelle section de `1-ruby.pdf` l'accompagne, et quand la lire.
2. Complétez le squelette `01_bases.rb`, question après question : les exercices s'écrivent tous dans le même fichier et s'appuient sur ce qui précède.
3. Lancez-le depuis `ruby/exercices/` avec `ruby 01_bases.rb`.
4. Certaines lignes lèvent volontairement une erreur, qui arrête le script : lisez le message, puis commentez la ligne pour que la suite s'exécute.

Pour essayer une ligne sans relancer tout le fichier, ouvrez IRB, la console de Ruby : `irb`. `irb -r ./01_bases.rb` charge en plus les méthodes et les classes du fichier (pas ses variables locales). `exit` pour sortir.

Une fois la correction publiée, comparez-la avec votre fichier : `ruby/corrections/01_bases.rb` s'exécute aussi, et donne les résultats attendus en commentaire (`# =>`).